

MixBytes()

[SECURITY AUDIT REPORT]

Lido L2 Direct Staking

PREPARED FOR LIDO

REPORT DATE

September 03, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	6
1.4 Security Audit Methodology	9
1.5 Risk Classification	11
1.6 Summary of Findings	12
2. Findings Report	12
2.1 Critical	12
2.2 High	12
2.3 Medium	13
2.4 Low	14
L-1 SyncTrigger.sweep() - missing zero-recipient guard permits silent fund burn	14
L-2 shouldSync() is not the on-chain executability predicate: syncs can silently stall	15
L-3 Owner fund-withdrawal paths emit no event: SyncTrigger.sweep and CREReceiver.withdrawETH	16
L-4 setFeeDtoO validates only a generic length, not the lane-specific return-fee format	17
3. About MixBytes	18

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Lido L2 Direct Staking lets users stake on an L2 (Optimism, Arbitrum, Base, Linea) and receive wstETH without manually bridging to Ethereum L1: user-supplied WETH accumulates in an L2 `OraclePool`, and an automation layer periodically syncs it - bridging the WETH to L1 via Chainlink CCIP, staking it into Lido, and bridging the resulting wstETH back to the originating L2 pool. The two in-scope contracts are that automation layer: `CREReceiver` is the authenticated entry point that accepts DON-signed reports from the Chainlink Keystone (CRE) forwarder and dispatches a single whitelisted, argument-less call, and `SyncTrigger` is the rate-limited decision/execution contract that decides when and how much to sync and fronts the CCIP native fee from its own balance. Both are non-upgradeable `Ownable` contracts with no permissionless state-mutating entry points; neither custodies user funds in steady state. This engagement covers the migration of the automation to Lido ownership and to Chainlink Runtime Environment (CRE) automation, replacing the retired Chainlink/Gelato automation.

The interim security review was conducted over 2.5 working days via manual code review and a proprietary AI-assisted analysis tool.

During the review, our focus covered both the core protocol logic and its interactions with external integrations - principally the upstream `chainlink-csr` `CustomSender` and the Chainlink CCIP and Keystone forwarder rails that the in-scope contracts call into. Alongside that, we worked through our internal checklist of common smart-contract weaknesses, including access control, reentrancy, arithmetic overflow/underflow and unsafe casts, rounding, low-level call handling, and input validation. The vectors most relevant to this specific protocol are listed below.

The upstream `chainlink-csr` contracts that the in-scope code calls - `CustomSender` (the `sync` entry point and CCIP send path), `OraclePool`, and the `FeeCodec` / `TokenHelper` libraries - were not part of this engagement; we reviewed only how the in-scope contracts invoke them and relied on their published source and interfaces. The Chainlink CCIP routers and the Keystone forwarder were likewise treated as a trusted external boundary rather than line-by-line audited.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Tight, single-purpose contracts with a clear trust model.** Both contracts are small, have zero permissionless state-mutating entry points, and enforce a layered authentication/authorization model (`onlyForwarder`, pinned report author, owner-allow-listed nullary call). The accounting in `SyncTrigger` was confirmed equivalent to the upstream `SyncAutomation` reference, with the adaptation adding a genuine bugfix and several setter guards rather than introducing regressions.
- **Hardening already applied at set-time.** The owner setters reject the dangerous configurations that would otherwise silently break automation at run time - `setForwarder(0)`, `setDelay(0)`, a `setFee0toD` exact-length and gas-floor check, and a `setFeeDto0` length check - which closes several self-DoS footguns present in the upstream template.
- **Residual risk is concentrated in owner-domain footguns and operations.** The findings that remain are low-severity and confined to trusted-owner actions: `SyncTrigger.sweep()` lacks the zero-recipient guard that its sibling `CREReceiver.withdrawETH()` has, and the native-fee float can be depleted to a point where syncs silently stall (a liveness, not a fund-loss, condition). We recommend adding the missing recipient guard and treating the fee float as an explicitly monitored operational invariant with a named-error pre-flight check.

We also note the following observations during the review. These are not defects in the two in-scope contracts but bear on safe deployment and operation; several are already documented in the project's docs:

- **ERC-165 interfaceId is load-bearing (verified correct).** The Keystone forwarder gates delivery on `supportsInterface(type(IReceiver).interfaceId)` and the ERC-165 base id. We confirmed `onReport(bytes,bytes)` has selector `0x805f2132`, that `IReceiver` declares only that function (so `type(IReceiver).interfaceId == 0x805f2132`), and that `CREReceiver.supportsInterface` returns true for `0x805f2132` and `0x01ffc9a7`. A one-selector change would silently brick report delivery with no revert; the current implementation is correct.
- **CRE forwarder address/ABI is not pinned on-chain.** `_forwarder` is set from an env value at deploy time with no `typeAndVersion` or ABI assertion, and the script-level `_expectedCREForwarder()` anchor currently returns `address(0)` (opt-out) on every lane. Each production forwarder must be confirmed per lane against Chainlink's directory before deployment, ensuring it is the ERC-165-gated `onReport(bytes,bytes)` variant with the `workflowId(32) | workflowName(10) | workflowOwner(20)` metadata layout.
- **DON-embedded workflowOwner versus registry owner.** `onReport` checks the DON-embedded author (`metadata[42:62]`) against the pinned `_expectedAuthor`. If the DON embeds a different address than expected, every report fails `InvalidAuthor` and syncs silently stall; the only positive proof is a live `CallExecuted` event.
- **The two payable `receive()` functions are the only permissionless-callable entry points, and serve different roles.** Neither contract holds user funds, and both are designed to carry a zero balance in steady state. `SyncTrigger.receive()` is functional and load-bearing: it accepts the CCIP fee refund that `CustomSender.sync` returns within the same transaction (without it, every sync would revert) and the permissionless native-token top-ups that replenish the fee float `triggerSync` spends - so `SyncTrigger` intentionally holds only its own operational float, recoverable by the owner via `sweep`. `CREReceiver.receive()` is defensive rather than functional: report dispatch (`target.call(data)`) forwards no value, so the receiver's balance is normally zero; the payable fallback exists only so that stray ETH is accepted rather than reverting, and is recoverable by the owner via `withdrawETH`. Both fallbacks are the intended, harmless float-funding/recovery path and do not expose user funds.
- **The fee float cannot be drained by an external actor (attack vector checked).** We specifically examined whether an attacker could empty `SyncTrigger`'s native fee float (or, on a `payInLink` lane, its LINK balance). It cannot. Every value-out path is gated: `sweep` is `onlyOwner`; the native fee spend in `triggerSync` is `onlyForwarder` (callable only by `CREReceiver`, which in turn requires the Keystone forwarder and `workflowOwner == _expectedAuthor`, and can dispatch only the single allow-listed nullary `triggerSync()`); and the deploy-time infinite LINK approval to `CustomSender` is exercised only inside `CustomSender.sync`, which is `onlyRole(SYNC_ROLE)` and is therefore reachable only through the same `triggerSync` path - not by calling `CustomSender` directly. Because each authorized sync is additionally bounded by the delay rate-limiter and the per-sync `maxFee` cap (and `_lastExecution` is advanced before the external call), the worst an authorized-but-spurious report can do is burn at most one `maxFee` per delay window; there is no permissionless or argument-controlled path to force-spend or extract the float. The only ways the float reaches zero are legitimate operation over time or the owner's own `sweep`.
- **Report replay (same-chain and cross-chain) was examined; cross-chain replay is possible, and is contained by the on-chain design rather than by signature domain separation.** The CRE forwarder does not mark a reverted transmission as used, so a failed report's signed blob stays permissionlessly re-submittable, and `CREReceiver.onReport` adds no timestamp/nonce idempotency guard of its own. Same-chain: a transmission is keyed `keccak256(receiver, workflowExecutionId, reportId)` and marked consumed once it succeeds, so only reports that already reverted can be re-submitted. Cross-chain: nothing in the signed material binds a report to a lane. `KeystoneForwarder.report()` is permissionless and its signature hash commits only to `keccak256(rawReport) || reportContext` - neither `receiver` nor `block.chainid`; `CREReceiver` checks only `workflowOwner`, `target` and `selector`, all of which are identical on the four lanes (same `CREReceiver` / `SyncTrigger` addresses, same `_expectedAuthor`, same `triggerSync()` selector, so the raw report `abi.encode(SyncTrigger, 0x340b2b0b)` is byte-identical); and because the four lanes are served by a single CRE workflow, the DON config and signer set that each forwarder validates against are shared as well, so the per-lane `InvalidConfig` / `InvalidSigner` rejection does not separate lanes. Transmission consumption is per-chain storage. An arbitrary account can therefore take the calldata of a report that executed on one lane and submit it to the other three forwarders, where it is accepted. We treat this as an accepted, bounded property rather than a vulnerability, because a replayed report can only do what the DON's own report for that lane would do: the sole allow-listed call is the nullary `triggerSync()`, whose amount is recomputed on-chain from the destination lane's pool balance, whose execution is gated by that lane's `delay` rate-limiter (`_lastExecution` is advanced before the external call) and `minAmount` / `maxAmount` bounds, and whose native fee spend is capped at one `maxFee` per delay window. A replay can at most make the one legitimate, already-due sync on the destination lane fire somewhat earlier

than the DON's next tick; it cannot manufacture extra syncs, choose the amount, fragment batches or drain the fee float (see the fee-float bullet above), and if the destination lane is not due it reverts with `SyncTriggerSyncNotNeeded` and stays harmlessly retryable. The cross-chain `chainSelector` guard the CRE docs recommend is therefore still not required for the current nullary design. The residual is forward-looking: if the allow-list is ever extended to a non-nullary or non-idempotent call, both a report-level idempotency guard and a lane binding (`chainSelector` / `block.chainid` in the signed report) would become load-bearing and should be added at that time.

- **Per-lane parameter differences are a chain-blindness footgun.** The four lanes are not interchangeable - Linea in particular uses a lower CCIP gas limit and a lower FeeQuoter cap, and applies a Gelato revoke during migration. A uniform parameter change applied across all lanes can pass everywhere except Linea. The non-atomic migration handover, run by an external party, should be kept short, especially the window in which the external owner still holds `ProxyAdmin` over the shared L1 receiver.
- **setDelay does not re-anchor the cooldown on activation (inherited from upstream).** `_setDelay` writes only `_delay` and never touches `_lastExecution`, while the sync gate is `block.timestamp >= _lastExecution + _delay`. Because `_lastExecution` is seeded to deploy time and only advanced by `triggerSync`, activating a long-dormant trigger (e.g. `setDelay(1 day)` 60 days after deploy) leaves the gate already open, so the first sync can fire immediately rather than one fresh `delay` window later. The behavior is identical to the upstream `SyncAutomation` reference, the impact is timing-only (a single, bounded, non-compounding fee spend at most one window early, on a sync that was due anyway; `triggerSync` then correctly rate-limits subsequent syncs), and immediate eligibility on activation is arguably desirable for draining accumulated WETH. If a fresh window on (re)activation is intended, re-anchor `_lastExecution = block.timestamp` inside `_setDelay`.
- **Dependency and compiler pinning is reproducible, with two provenance caveats.** All dependencies are managed as git submodules (no soldeer/npm lockfile) and are recorded at the audited commit as exact gitlink pins and the compiler is pinned to `solc 0.8.34` / `evm_version = osaka`, matching `pragma solidity 0.8.34` in both in-scope files with no caret range, so the build is fully reproducible. Two items are worth noting: `.gitmodules` declares `branch = main` for `chainlink-csr`, so a `git submodule update --remote` would silently fast-forward the most audit-relevant dependency off its pinned commit - we recommend removing the `branch` line or pinning it to a tag; and `chainlink-csr` is pinned to an untagged commit of a third-party fork (<https://github.com/Aphyla/chainlink-csr>) rather than a tagged upstream release, which is the dependency that supplies the in-scope fee/native accounting (`FeeCodec`, `TokenHelper`, `CustomSender`, `SyncAutomation`) and is not known to be audited.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	L2 Direct Staking
TYPE	Solidity
PLATFORM	EVM
TIMELINE	11.06.2026 - 04.08.2026

Scope of Audit

`src/cre/CREReceiver.sol``src/SyncTrigger.sol`

Versions Log

DATE	COMMIT HASH	NOTE
11.06.2026	1838eb5fb177ec775b268812ae4a9ee5b5d7a62c	Initial Commit
22.06.2026	27dead7488961fa74f47da2687b2d94dd4f83976	Commit for Re-audit
04.08.2026	3d1d484c31d39881cdb8423614e3043e89e027f	Commit with Updates

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
SyncTrigger.sol	0x871a5cddE9813627Ff37A2895A0c9B117A664622	Arbitrum
CREReceiver.sol	0x09BdB4E8BA68d245DCb1c6fbEb1e4f13b57cc69A	Arbitrum
SyncTrigger.sol	0x871a5cddE9813627Ff37A2895A0c9B117A664622	Optimism
CREReceiver.sol	0x09BdB4E8BA68d245DCb1c6fbEb1e4f13b57cc69A	Optimism
SyncTrigger.sol	0x871a5cddE9813627Ff37A2895A0c9B117A664622	Base
CREReceiver.sol	0x09BdB4E8BA68d245DCb1c6fbEb1e4f13b57cc69A	Base
SyncTrigger.sol	0x871a5cddE9813627Ff37A2895A0c9B117A664622	Linea
CREReceiver.sol	0x09BdB4E8BA68d245DCb1c6fbEb1e4f13b57cc69A	Linea

Both in-scope contracts are deployed at the same address on all four lanes. The deployment blocks are: Arbitrum `489061828` / `489061820`, Optimism `154852960` / `154852959`, Base `49284030` / `49284030`, Linea `31560854` / `31560854` (SyncTrigger / CREReceiver respectively).

The deployment compiles byte-for-byte from the audit-scope source at commit `3d1d484c31d39881cdbe8423614e3043e89e027f`.

Constructor immutables and stored parameters were read back from the on-chain getters on every lane and reconciled against the client's expected-state configuration (`config/state/*.inputs.yaml`); all values match.

The parameters are adequate and internally consistent. `delay = 12h` sits far above the `MIN_DELAY = 1 minute` floor, so the rate-limiter is conservative; `minAmount = 5 WETH` / `maxAmount = 100 WETH` bound each sync sensibly against the 12h accumulation window. The Linea lane correctly carries the lower `maxGasLimit` (3M vs 7M) and the lower `fee0toD` gasLimit (500k vs 1M) that its CCIP `EVM2EVMOnRamp` cap requires, and its 17-byte all-zero `feeDto0` blob is the correct Linea encoding - this is exactly the per-lane divergence flagged as a chain-blindness footgun in section 1.2, and it is configured correctly here. Every `fee0toD` encodes `payInLink = false`, consistent with the contract rejecting LINK payment outright. `MIN_DELAY` reads back as `60` on all lanes, confirming the constant is live in the deployed bytecode.

Wiring is correct and complete on all four lanes: each `SyncTrigger.getForwarder()` points at that lane's `CREReceiver` (`0x09BdB4E8BA68d245DCb1c6fbEb1e4f13b57cc69A`); each `CREReceiver` has the Chainlink Keystone forwarder set - `0xF8344CFd5c43616a4366C34E3EEE75af79a74482` on Arbitrum, Optimism and Base, and `0x9eF6468C5f37b976E57d52054c693269479A784d` on Linea - and allow-lists exactly the nullary `SyncTrigger.triggerSync()` selector `0x340b2b0b` on its own lane's trigger (`0x871a5cddE9813627Ff37A2895A0c9B117A664622`); and each `SyncTrigger` holds `SYNC_ROLE` (`0xbb1ef2b79fa8154a13ffa50bd30e5f91ed93ff9b924bd04be671240cbc9d4b71`) on its `CustomSender` - `0x72229141D4B016682d3618ECe47c046f30Da4AD1` on Arbitrum, and `0x328de900860816d29D1367F6903a24D8ed40C997` on Optimism, Base and Linea.

Two observations on the deployed configuration:

- **Ownership is a single EOA, not a multisig or the DAO.** On all four lanes `SyncTrigger.owner()`, `CREReceiver.owner()` and `CREReceiver.getExpectedAuthor()` are the same externally-owned account `0xBdF111fec2e818Ad9c76fbBaE46144746AD55773`, which is also the deployer of all eight contracts. We confirmed it is a plain EOA rather than a smart-contract wallet: `eth_getCode` returns `0x` on all four lanes and on Ethereum mainnet. There is consequently no multisig, module, or guard layer behind it. This is the client's intended `L2AutomationOwner` and is consistent across lanes, so it is not a misconfiguration - but it does concentrate the entire owner domain (`setForwarder`, `setDelay`, `setAmounts`, `setFee0toD` / `setFeeDto0`, `setMaxGasLimit`, `sweep`) plus the DON-author identity in one key, with no threshold and no timelock. The blast radius is bounded - neither contract custodies user funds, and the worst case is automation misconfiguration or sweeping the operational float, not loss of staked assets - but a key compromise would stall or misdirect the automation on every lane simultaneously. We recommend migrating these owners to a multisig (or the DAO's L2 governance executor) once the migration has settled, and note that the same key doubling as `_expectedAuthor` means a rotation must be coordinated across both roles.
- **The native fee float has no headroom, and Arbitrum is already below one sync.** `SyncTrigger` balances were ~ 0.1065 ETH (Arbitrum), ~ 0.1295 ETH (Optimism), 0.13 ETH (Base) and ~ 0.1291 ETH (Linea), against a worst-case `getMaxFees()` of 0.126005 ETH on Arbitrum and 0.125 ETH elsewhere - so Arbitrum reads `canSync() == false` and the rest cover only $\sim 1.03\times$ one sync (in practice more, since the unused CCIP fee is refunded in-transaction). This is latent: `shouldSyncAmount()` is `0` on all four lanes because every `OraclePool` is empty (~ 0.0003 WETH on Base, `0` elsewhere) against a `minAmount` of `5 WETH`, with the 12h delay long elapsed and no pool paused - the lanes are idle for lack of deposits, so the shortfall surfaces only once Arbitrum accumulates `5 WETH`. It is not a fund-loss condition (`sweep` recovers the float, `triggerSync` fails closed via `SyncTriggerInsufficientFloat`, and `canSync() == false` makes the DON suppress the report), but it confirms the liveness concern in section 1.2. Top up Arbitrum before relying on that lane, and monitor the float with alerting set well above one `maxFee`.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

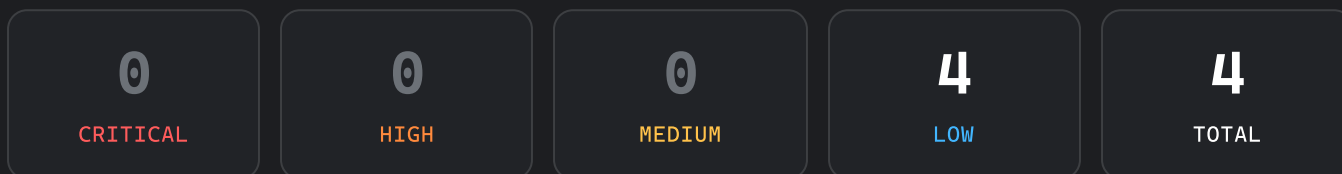
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
L-1	SyncTrigger.sweep() - missing zero-recipient guard permits silent fund burn	● LOW	● FIXED
L-2	shouldSync() is not the on-chain executability predicate: syncs can silently stall	● LOW	● FIXED
L-3	Owner fund-withdrawal paths emit no event: SyncTrigger.sweep and CREReceiver.withdrawETH	● LOW	● FIXED
L-4	setFeeDto0 validates only a generic length, not the lane-specific return-fee format	● LOW	● ACKNOWLEDGED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 SyncTrigger.sweep() - missing zero-recipient guard permits silent fund burn

● LOW

● FIXED

DESCRIPTION

`SyncTrigger.sweep(address token, address recipient, uint256 amount)` is an owner-only recovery function that delegates directly to `TokenHelper.transfer(token, recipient, amount)` without validating that `recipient != address(0)`.

For the native-token branch (`token == address(0)`), `TokenHelper.transferNative` executes `recipient.call{value: amount}("")` and only checks the returned `success` flag. A call to `address(0)` targets an account with no code and therefore succeeds, so `sweep(address(0), address(0), amount)` silently transfers the native balance to the zero address, permanently burning it with no revert. The repository's own test suite documents this EVM behavior ("a call to `address(0)` succeeds (no code)").

This is a validation asymmetry within the same codebase: the analogous owner-only recovery function `CREReceiver.withdrawETH(to, amount)` was hardened with `if (to == address(0)) revert InvalidRecipientAddress();`, but `SyncTrigger.sweep` was not.

This issue is classified Low severity because, although the in-isolation impact is fund loss, the trigger is exclusively an owner mistake with no external actor or market condition (Low likelihood), and it concerns only the contract's own fee float rather than user funds.

RECOMMENDATION

We recommend adding a zero-recipient guard to `sweep`, mirroring `CREReceiver.withdrawETH` - for example, reverting with a dedicated error such as `SyncTriggerInvalidRecipient` when `recipient == address(0)` before delegating to `TokenHelper.transfer`.

L-2 `shouldSync()` is not the on-chain executability predicate: syncs can silently stall

● LOW

● FIXED

DESCRIPTION

The CRE workflow uses `shouldSync()` as its trigger predicate, but it checks only two conditions via `_getAmountToSync` - the `delay` has elapsed and the pool's `WNATIVE` balance is `>= minAmount`. The actual success of `triggerSync` $\rightarrow$ `CustomSender.sync` depends on further on-chain preconditions that `shouldSync()` never reads. Whenever one is unmet, the predicate still returns `true`, the DON keeps submitting a report every cron tick, and each `triggerSync` reverts - with no on-chain signal. WETH accumulates in the L2 pool and is never staked.

The most material case arises structurally, with no external action: `triggerSync` fronts the CCIP native fee from its own balance (`sync{value: nativeAmount}`), and there is no pre-call balance check. Each sync consumes roughly the actual CCIP fee while only the `maxFee0toD` overage is refunded to `receive()`, so the fee float depletes monotonically with use; once `address(this).balance < nativeAmount`, every sync reverts at the value transfer (a bare EVM error surfaced as `CallExecutionFailed`) and syncing stalls indefinitely until the float is replenished. Other preconditions invisible to the predicate include a revoked `SYNC_ROLE` on `CustomSender`, a paused `OraclePool`, a live CCIP fee exceeding the configured `maxFee`, and a `fee0toD` gas limit above the lane's FeeQuoter cap - each of which likewise reverts the sync while `shouldSync()` stays `true`. The gas-limit case is itself a set-time footgun: `_setFee0toD` enforces only the lower floor (`gasLimit >= MIN_PROCESS_MESSAGE_GAS`) and no upper bound, so a chain-blind config push (e.g. a uniform `gasLimit = 5_000_000` raised across all lanes) is accepted on every lane yet exceeds the lower per-lane cap on Linea, after which every `triggerSync` reverts inside `sync` with no on-chain signal.

This issue is classified Low severity: it is not attacker-triggered and no funds are lost (WETH remains in the pool), but a core protocol function can be silently stalled (indefinitely, in the fee-float case) with no on-chain indication, requiring out-of-band operator intervention to detect and restore.

RECOMMENDATION

We recommend making the trigger predicate a sound approximation of on-chain executability rather than just `delay` + balance: surface the additional preconditions through a richer `canSync()` view (and/or the off-chain workflow predicate) that also checks the native/LINK fee float against the implied per-sync cost, `hasRole(SYNC_ROLE)`, `!pool.paused()`, and live-fee adequacy, so the DON suppresses guaranteed-revert reports. At minimum, add a pre-flight balance check in `triggerSync` that reverts with a named error (e.g. `SyncTriggerInsufficientFloat`) and document the float as an explicit liveness invariant with off-chain low-balance alerting keyed to `getMaxFees().maxNativeFee`. For the gas-limit case specifically, add a per-lane `maxGasLimit` upper bound (seeded to the lane's FeeQuoter `maxPerMsgGasLimit` at deploy) and reject `gasLimit > maxGasLimit` in `_setFee0toD`, so a chain-blind over-bump fails loudly at set time rather than silently inside `sync`.

L-3 Owner fund-withdrawal paths emit no event: `SyncTrigger.sweep` and `CREReceiver.withdrawETH`

● LOW

● FIXED

DESCRIPTION

Both in-scope contracts expose an owner-only function that moves value out, and neither emits an event:

- `SyncTrigger.sweep(token, recipient, amount)` forwards to `TokenHelper.transfer` and emits nothing.
- `CREReceiver.withdrawETH(to, amount)` performs `to.call{value: amount}("")` and emits nothing.

This is an asymmetry with the rest of both contracts, where every other state-changing or value-relevant action is logged: all setters emit (`ForwarderSet` / `DelaySet` / `AmountsSet` / `Fee0toDSet` / `FeeDto0Set` in `SyncTrigger` ; `ForwarderUpdated` / `ExpectedAuthorUpdated` / `AllowedCallUpdated` in `CREReceiver`), and report dispatch emits `CallExecuted` . The two withdrawal paths are the lone exceptions.

This issue is classified Low severity: no funds are at risk from the omission itself, but owner withdrawals of the fee float and of the receiver's ETH produce no first-class on-chain trail, degrading monitoring and incident response (e.g. detecting an erroneous sweep, or one made via a compromised owner key).

RECOMMENDATION

We recommend declaring and emitting an event in each withdrawal path - e.g. `emit Swept(token, recipient, amount)` in `SyncTrigger.sweep` and `emit ETHWithdrawn(to, amount)` in `CREReceiver.withdrawETH` - so that every value-moving owner action leaves an auditable trail consistent with the rest of both contracts.

L-4 `setFeeDto0` validates only a generic length, not the lane-specific return-fee format

● LOW

● ACKNOWLEDGED

DESCRIPTION

The `feeDto0` value encodes the L1->L2 return-bridge fee, and its required structure is lane-specific. The L1 adapter for each destination decodes the same blob with a strict, exact-length decoder plus additional invariants: Arbitrum `decodeArbitrumL1toL2` requires exactly 29 bytes (carrying `maxGas` and `gasPriceBid`) with `payInLink == false`; Optimism and Base `decode{Optimism,Base}L1toL2` require exactly 21 bytes with `payInLink == false` and `feeAmount == 0`; Linea `decodeLineaL1toL2` requires exactly 17 bytes with `payInLink == false` and `feeAmount == 0`.

`SyncTrigger._setFeeDto0`, however, validates with the generic `FeeCodec.decodeFee`, which enforces nothing beyond `length >= 17` and reads only the universal `feeAmount(16) | payInLink(1)` prefix. Because `SyncTrigger` is not lane-aware, none of the bridge-specific invariants are checked on L2, so the setter accepts a misconfigured blob - a wrong length (e.g. a 21-byte Optimism encoding mistakenly set on the Arbitrum lane), a non-zero `payInLink`, or a non-zero `feeAmount` on a zero-fee lane - all of which pass both the set-time check and the L2 source leg silently.

The failure is asynchronous and moves funds before surfacing. `triggerSync` succeeds: `_lastExecution` advances, WETH is pulled out of the L2 `OraclePool`, and the funds are bridged to L1. Only on L1 does the adapter's `_sendToken` revert inside the strict decoder. Because the L1 receiver is defensive (`try this.processMessage catch`), the revert is caught and the message is stored as failed, leaving the bridged WETH parked at the L1 receiver, unstaked. `retryFailedMessage` is futile - the malformed `feeDto0` is frozen in the original message and the decode reverts deterministically on every replay - so recovery requires the privileged `recoverTokens` (`DEFAULT_ADMIN_ROLE`) on L1 plus a corrected `setFeeDto0` on L2.

This contrasts with the sibling setter `_setFee0toD`, which validates strictly - it decodes with `FeeCodec.decodeCCIP` (exact 21 bytes) and enforces the gas floor - precisely because `fee0toD` is always the CCIP format, whereas `feeDto0`'s format varies by lane.

This issue is classified Low severity: it is reachable only through a privileged owner/operator misconfiguration rather than an external attacker, and the bridged funds are parked and recoverable by L1 governance rather than permanently lost - but the impact is a per-lane liveness outage that strands WETH at the L1 receiver and requires manual cross-layer intervention to clear. It is a concrete instance of the per-lane chain-blindness footgun noted in the Executive Summary.

RECOMMENDATION

We recommend enforcing the destination bridge's structural invariants at set time in `_setFeeDto0`, mirroring how `_setFee0toD` already pins the exact CCIP layout via `decodeCCIP`. Concretely, record the expected `feeDto0` length for the target lane as a per-lane immutable at construction (the deploy script already knows the lane), assert that exact length, and reject `payInLink == true` - and, for the zero-fee lanes (Optimism/Base/Linea), `feeAmount != 0` - so that a malformed configuration reverts immediately on L2 instead of failing asynchronously on L1 after WETH has already been bridged. If that is too invasive, document explicitly that the `>= 17` check is only a lower bound and have the deploy/config tooling encode and verify the correct per-lane `feeDto0` against the lane's adapter before the trigger is armed.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io